

♦ PROJETO – INTRODUÇÃO AO MACHINE LEARNING

O objetivo desse projeto é aplicar o processo completo de análise de dados e Machine Learning em um conjunto de dados real, envolvendo a compreensão do dataset, a realização de análise exploratória, a preparação dos dados, o treinamento de diferentes modelos e a avaliação de seus desempenhos, com o objetivo de comparar os resultados obtidos e selecionar o modelo mais adequado com base em critérios de desempenho, estabilidade e interpretação dos resultados.

Cada etapa deve ter:

- evidência (resultado/print)
- respostas às perguntas

Na entrega final:

- Incluir links para os notebooks criados para realização do projeto

ETAPAS

♦ ETAPA 1 – Seleção e compreensão dos dados.....	2
♦ ETAPA 2 – Análise Exploratória dos Dados (EDA).....	5
♦ ETAPA 3 – Preparação dos dados.....	8
♦ ETAPA 4 – Modelagem.....	11
♦ ETAPA 5 – Testes e avaliação dos modelos.....	14
♦ ETAPA 6 – Conclusão e recomendação.....	17
Links para os notebooks.....	20

♦ ETAPA 1 – Seleção e compreensão dos dados

🎯 Objetivo

Compreender o conjunto de dados, identificar o problema de análise e definir a variável alvo.

📋 Passos

1. Escolher o dataset

- Selecionar um dataset
- Veja os sites a seguir e escolha um dataset:
 - <https://www.kaggle.com/datasets>
 - <https://archive.ics.uci.edu/datasets> (diversificado, mas está fora do ar frequentemente)
 - <https://data.gov/>
 - <https://data.worldbank.org/>
 - <https://datasetsearch.research.google.com/>
 - <https://www.openml.org/>

Observação: não usar os datasets já usados em aula!

2. Carregar os dados

- Ler o arquivo para um DataFrame (ex: CSV)

3. Visualizar os dados iniciais

Utilizar funções para entender a estrutura:

- `head()` → visualizar as primeiras linhas

```
arquivos = os.listdir(path)

arquivo_csv = [f for f in arquivos if f.endswith('.csv')][0]

try:
    df = pd.read_csv(os.path.join(path, arquivo_csv), encoding='latin1', sep=';')
    print("Carregado com sucesso usando separador ';'")
except:
    df = pd.read_csv(os.path.join(path, arquivo_csv), encoding='latin1', sep=',', on_bad_lines='skip')
    print("Carregado com sucesso usando usando ','")

display(df.head())
```

/tmp/ipykernel_10826/1720960333.py:8: DtypeWarning: Columns (10,21,22) have mixed types. Specify dtype option on import or set low_memory

df = pd.read_csv(os.path.join(path, arquivo_csv), encoding='latin1', sep=',', on_bad_lines='skip')

Carregado com sucesso usando usando ','

	TP_NOT	ID_AGRAVO	DT_NOTIFIC	SEM_NOT	MU_ANO	SG_UF_NOT	ID_MUNICIP	ID_REGIONA	DT_SIN_PRI	SEM_PRI	...	COM_SISTEM	COM_RENAL	COM_
0	2	X29	20190101	201901	2019	35	351880	1333.0	20190101	201901	...	2.0	NaN	
1	2	X29	20190101	201901	2019	26	261000	1498.0	20181231	201901	...	2.0	NaN	
2	2	X29	20190101	201901	2019	26	261000	1498.0	20181231	201901	...	2.0	NaN	
3	2	X29	20190101	201901	2019	26	261000	1498.0	20181231	201901	...	2.0	NaN	
4	2	X29	20190101	201901	2019	26	261000	1498.0	20190101	201901	...	2.0	NaN	

5 rows x 15 columns

- `info()` → verificar tipos de dados e valores não nulos

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1335648 entries, 0 to 1335647
Data columns (total 76 columns):
#   Column                Non-Null Count  Dtype
---  -
0   TP_NOT                 1335648 non-null  int64
1   ID_AGRAVO              1335648 non-null  object
2   DT_NOTIFIC             1335648 non-null  int64
3   SEM_NOT                1335648 non-null  int64
4   NU_ANO                 1335648 non-null  int64
5   SG_UF_NOT              1335648 non-null  int64
6   ID_MUNICIP             1335648 non-null  int64
7   ID_REGIONA            1267106 non-null  float64
8   DT_SIN_PRI             1335648 non-null  int64
9   SEM_PRI                1335648 non-null  int64
10  ANO_NASC               1318332 non-null  object
11  NU_IDADE_N             1335648 non-null  int64
12  CS_SEXO                 1335648 non-null  object
13  CS_GESTANT              1335605 non-null  float64
14  CS_RACA                 1317763 non-null  float64
15  CS_ESCOL_N             1202586 non-null  float64
16  SG_UF                   1335359 non-null  float64
17  ID_MN_RESI             1335359 non-null  float64
18  ID_RG_RESI             1267106 non-null  float64
19  ID_PAIS                 1335644 non-null  float64
20  DT_INVEST              1335648 non-null  int64
21  ID_OCUPA_N             700654 non-null   object
22  ANT_DT_ACT              1335648 non-null  object
23  ANT_UF                 1335578 non-null  float64
24  ANT_MUNIC_             1335577 non-null  float64
25  ANT_LOCALI_            0 non-null        float64
26  ANT_TEMPO_             1302779 non-null  float64
27  ANT_LOCA_1             1335578 non-null  float64
28  MCLI_LOCAL             1327214 non-null  float64
29  CLI_DOR                1244885 non-null  float64
30  CLI_EDEMA              1244769 non-null  float64
31  CLI_EQUIMO             1244703 non-null  float64
32  CLI_NECROS             1244694 non-null  float64
33  CLI_LOCAL_             1244646 non-null  float64
34  CLI_LOCA_1             138347 non-null   object
35  MCLI_SIST              1309439 non-null  float64
36  CLI_NEURO              78007 non-null    float64
37  CLI_URTI              78008 non-null    float64
```

- `describe()` → obter estatísticas básicas (para colunas numéricas)

```
display(df.describe())
```

	TP_NOT	DT_NOTIFIC	SEM_NOT	NU_ANO	SG_UF_NOT	ID_MUNICIP	ID_REGIONA	DT_SIN_PRI	SEM_PRI	NU_IDADE_N
count	1335648.0	1.335648e+06	1.335648e+06	1.335648e+06	1.335648e+06	1.335648e+06	1.267106e+06	1.335648e+06	1.335648e+06	1.335648e+06
mean	2.0	2.020047e+07	2.019225e+05	2.019980e+03	3.122538e+01	3.140185e+05	1.570383e+03	2.020041e+07	2.019220e+05	4.015931e+03
std	0.0	1.422389e+04	4.522515e+03	1.422004e+00	8.835643e+00	8.862478e+04	8.560615e+02	1.429127e+04	4.522530e+03	1.858354e+03
min	2.0	2.018010e+07	1.801000e+03	2.018000e+03	1.100000e+01	1.100010e+05	1.331000e+03	1.939100e+07	1.801000e+03	1.200000e+03
25%	2.0	2.019031e+07	2.019110e+05	2.019000e+03	2.800000e+01	2.810300e+05	1.377000e+03	2.019031e+07	2.019110e+05	4.020000e+03
50%	2.0	2.020052e+07	2.020210e+05	2.020000e+03	3.100000e+01	3.130400e+05	1.460000e+03	2.020052e+07	2.020210e+05	4.035000e+03
75%	2.0	2.021102e+07	2.021410e+05	2.021000e+03	3.500000e+01	3.536000e+05	1.512000e+03	2.021102e+07	2.021410e+05	4.052000e+03
max	2.0	2.022123e+07	2.022520e+05	2.022000e+03	5.300000e+01	5.300100e+05	8.256000e+03	2.022123e+07	2.022520e+05	4.130000e+03

8 rows x 11 columns

4. Identificar dimensões do dataset

- Verificar:
 - número de linhas
 - número de colunas

```
linhas, colunas = df.shape
print(f"O dataset possui {linhas} linhas e {colunas} colunas.")

*** O dataset possui 1335648 linhas e 76 colunas.
```

5. Analisar as colunas

Para cada coluna, observar:

- nome
- tipo de dado:
 - numérico (int, float)
 - categórico (texto, categorias)

```
print("Contagem de tipos de dados:")
print(df.dtypes.value_counts())

colunas_numericas = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
colunas_categoricas = df.select_dtypes(include=['object']).columns.tolist()

print(f"n Numéricas: {colunas_numericas[:10]}")
print(f"n Categóricas: {colunas_categoricas[:10]}")

*** Contagem de tipos de dados:
float64    46
int64      28
object     18
Name: count, dtype: int64
n Numéricas: ['TP_NOT', 'DT_NOTIFIC', 'SEM_NOT', 'NU_ANO', 'SG_UF_NOT', 'ID_MUNICIP', 'ID_REGIONA', 'DT_SIN_PRI', 'SEM_PRI', 'NU_IDADE_N']
Categóricas: ['ID_AGRAVO', 'ANO_NASC', 'CS_SEXO', 'ID_OCUPA_N', 'ANT_DT_ACI', 'CLI_LOCA_1', 'CLI_OUTR_3', 'ANI_TIPO_1', 'DT_OBITO', 'DT_E']
```

6. Identificar a variável alvo (target)

Escolher uma coluna que será prevista pelo modelo.

👉 Dica:

- variável alvo deve representar o “resultado” que você quer prever
- exemplos:
 - “aprovado/reprovado” → classificação
 - “preço” → regressão

7. Definir o tipo de problema

Classificar o problema como:

- **Classificação** → quando a variável alvo é categórica (ex: sim/não, classes, categorias)
- **Regressão** → quando a variável alvo é numérica contínua (ex: valor, quantidade, nota)

8. Formular uma hipótese inicial

Com base nos dados observados:

- pensar em uma possível relação entre variáveis
- levantar uma ideia do tipo:
 - “A variável X influencia Y”

Análise (responder com suas palavras)

1. Qual variável você escolheu como alvo?

R: Variável TRA_CLASSI. No dataset, ela representa a classificação da gravidade do acidente.

2. Por que essa variável foi escolhida?

R: Foi escolhida porque o objetivo é identificar os fatores que levam um acidente com animal peçonhento a ser mais ou menos grave.

3. O problema é classificação ou regressão? Explique com base no tipo de dado.

R: O problema é a Classificação. Isso ocorre porque a variável alvo é categórica. Ela não representa um valor numérico contínuo, mas sim classes específicas codificadas com números.

4. Quantas linhas e colunas o dataset possui?

R: 1.335.648 linhas e 76 colunas

5. Qual variável você acredita que mais influencia o resultado? Por quê?

R: A variável TP_ACIDENT é a que mais influencia o resultado. Isso porque o potencial de envenenamento varia drasticamente entre as espécies.

6. Qual é a sua hipótese inicial sobre os dados?

R: É que existe uma correlação direta entre o Tipo de Animal e a Idade da Vítima na definição da gravidade do caso. Acredito que o modelo mostrará que crianças e idosos apresentam quadros mais graves em acidentes comparado com adultos.

 Entrega da etapa

- Print das funções (`head()`, `info()`, `describe()`)
- Respostas das perguntas

♦ ETAPA 2 – Análise Exploratória dos Dados (EDA)

Objetivo

Explorar os dados para identificar padrões, relações entre variáveis e possíveis problemas antes da modelagem.

Passos

1. Revisar os tipos de variáveis

- Separar mentalmente (ou anotar):
 - variáveis numéricas
 - NU_IDADE_N: Idade da vítima
 - NU_AMPOLAS: Quantidade de ampolas de soro utilizadas
 - ANT_TEMPO: Tempo entre o acidente e o atendimento
 - variáveis categóricas
 - SG_UF_NOT: O estado
 - TP_ACIDENT: Tipo de animal
 - CS_SEXO: Gênero da vítima
 - TRA_CLASSI: Variável Alvo
 - EVOLUCAO: Desfecho clínico

👉 Isso é importante para decidir quais gráficos usar

2. Analisar variáveis numéricas (distribuição)

Para cada variável numérica:

- visualizar a distribuição usando:
 - `hist()` ou `histplot()`

Observar:

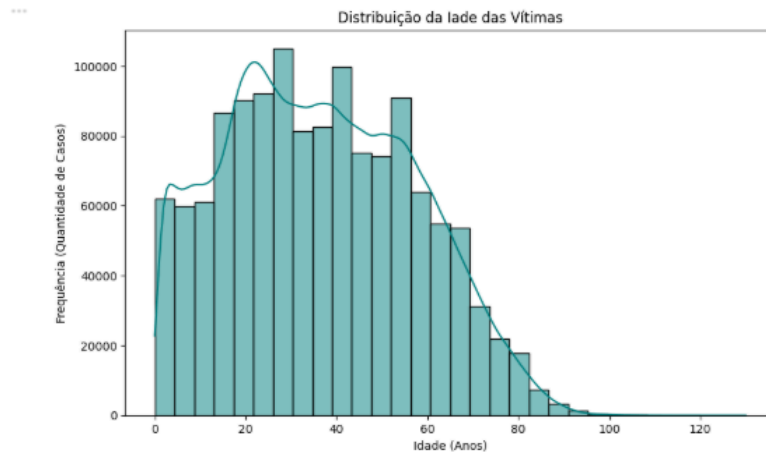
- valores mais frequentes
- dispersão dos dados
- possíveis valores extremos

```
idade_corrigida = df[df['NU_IDADE_N'] >= 4000]['NU_IDADE_N'] - 4000

plt.figure(figsize=(10,6))

sns.histplot(idade_corrigida, bins=30, kde=True, color='teal')

plt.title('Distribuição da Idade das Vítimas')
plt.xlabel('Idade (Anos)')
plt.ylabel('Frequência (Quantidade de Casos)')
plt.show()
```



3. Analisar variáveis categóricas

Para variáveis categóricas:

- contar frequência de cada categoria usando:
 - `value_counts()`
- visualizar com:
 - gráfico de barras (`barplot()` ou similar)

Observar:

- categorias mais comuns
- categorias raras

```
plt.figure(figsize=(12,5))

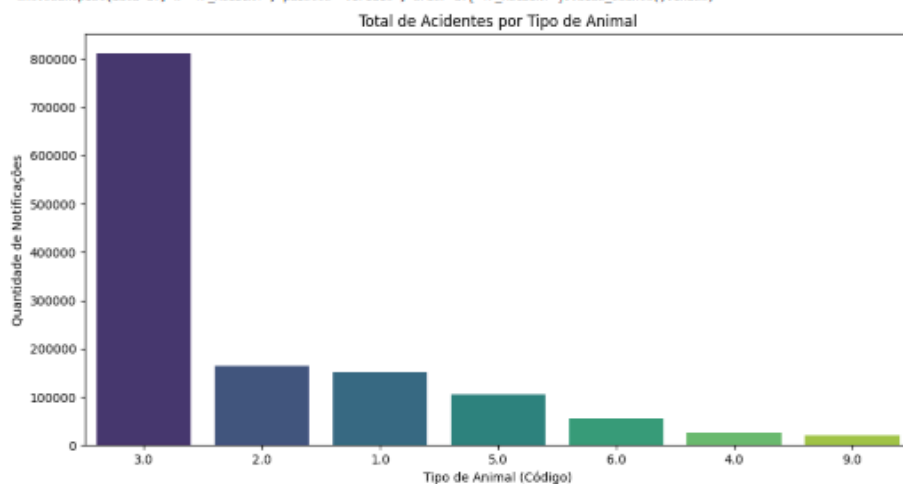
sns.countplot(data=df, x='TP_ACIDENT', palette='viridis', order=df['TP_ACIDENT'].value_counts().index)

plt.title('Total de Acidentes por Tipo de Animal')
plt.xlabel('Tipo de Animal (Código)')
plt.ylabel('Quantidade de Notificações')
plt.show()
```

*** /tmp/ipykernel_65886/2714733706.py:3: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.countplot(data=df, x='TP_ACIDENT', palette='viridis', order=df['TP_ACIDENT'].value_counts().index)
```



4. Analisar relação com a variável alvo

Escolher pelo menos 2 variáveis e comparar com a variável alvo:

- numérica vs numérica:
 - usar `scatterplot()`
- categórica vs alvo:
 - usar `boxplot()` ou gráfico de barras

Observar:

- se há tendência (aumenta/diminui junto)
- se há separação entre grupos

```
plt.figure(figsize=(12, 5))
```

```
sns.countplot(data=df, x='TRA_CLASSI', palette='viridis')
plt.title('Frequência das Categorias de Gravidade')
plt.xlabel('Gravidade (1-Leve, 2-Moderado, 3-Grave)')
plt.ylabel('Quantidade')
plt.show()
```

```
*** Contagem por Gravidade
```

```
TRA_CLASSI
```

```
1.0    1115403
```

```
2.0    137200
```

```
3.0     33679
```

```
4.0     18827
```

```
Name: count, dtype: int64
```

```
/nContagem por Estado - Top 10:
```

```
SG_UF_NOT
```

```
31    248065
```

```
35    244610
```

```
29    129526
```

```
26    105685
```

```
41     81043
```

```
27     61242
```

```
23     44601
```

```
15     43701
```

```
52     39967
```

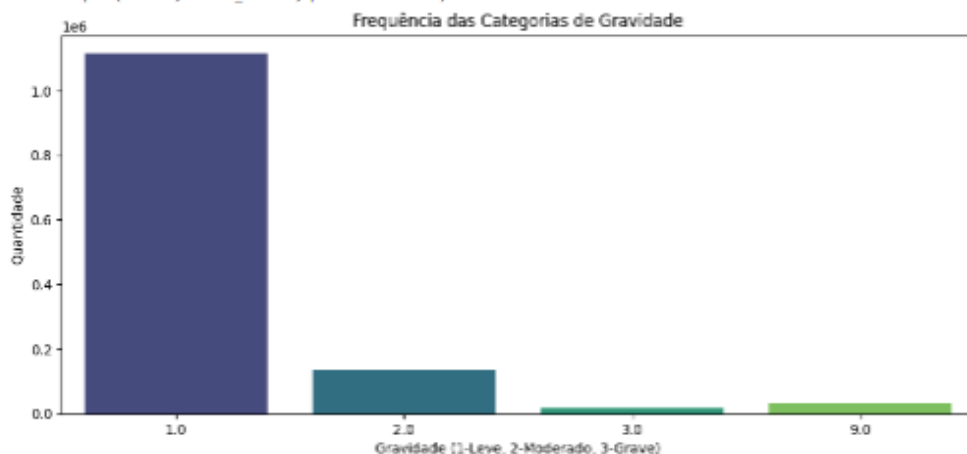
```
42     37658
```

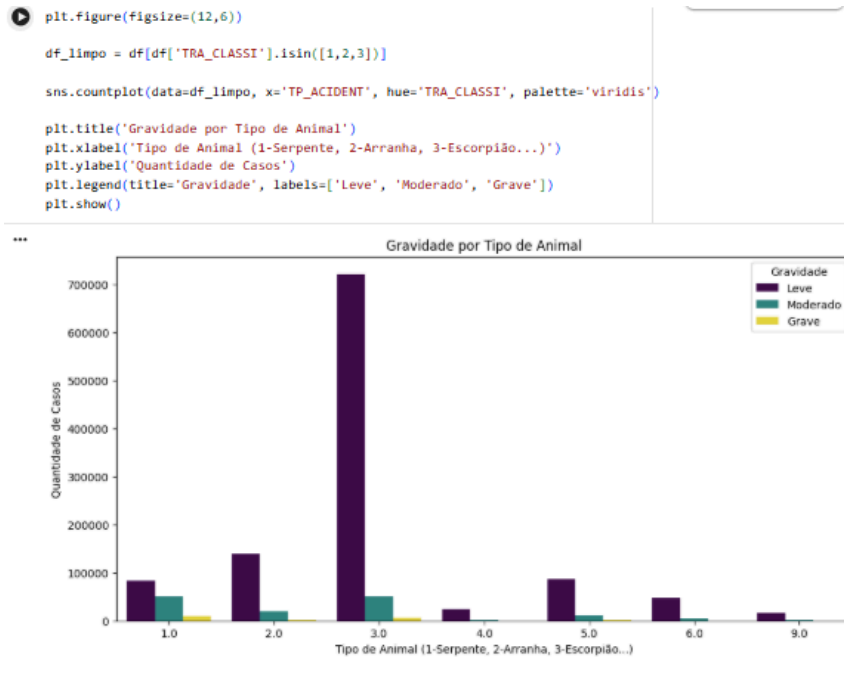
```
Name: count, dtype: int64
```

```
/tmp/ipykernel_65886/2809034356.py:9: FutureWarning:
```

```
Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.
```

```
sns.countplot(data=df, x='TRA_CLASSI', palette='viridis')
```





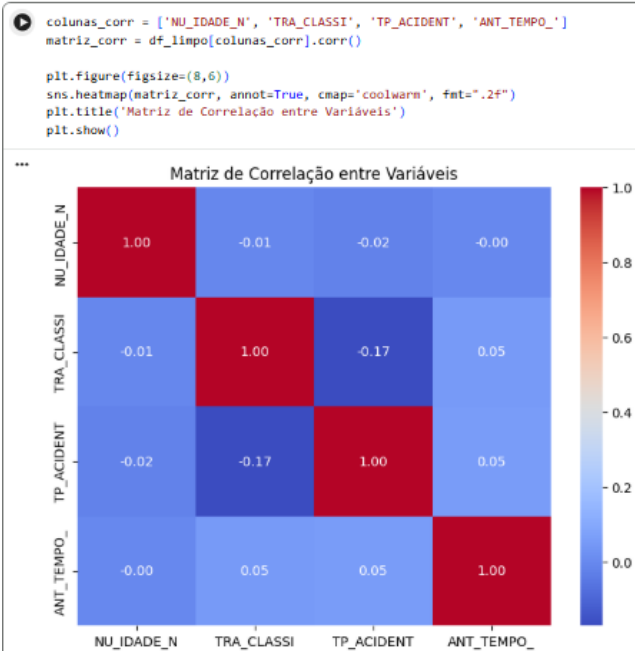
5. Analisar correlação (quando aplicável)

Para variáveis numéricas:

- calcular correlação usando:
 - `corr()`
- visualizar (opcional):
 - heatmap

Observar:

- valores próximos de:
 - 1 → forte relação positiva
 - -1 → forte relação negativa
 - 0 → pouca relação

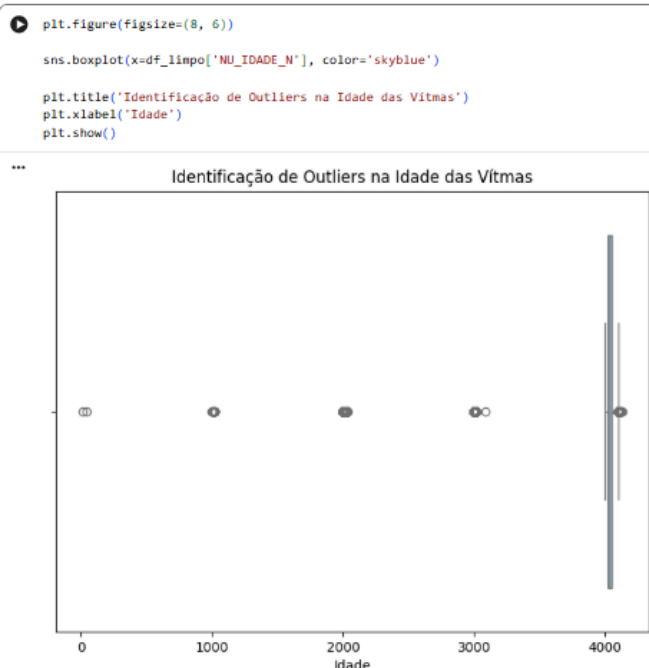


6. Identificar outliers (valores extremos)

- usar:
 - `boxplot()`

Observar:

- valores muito distantes do restante dos dados



Análise (responder com suas palavras)

1. Como estão distribuídos os dados das variáveis numéricas?

R: Os dados da variável `NU_IDADE_N` apresentam uma distribuição multimodal e estão concentrados principalmente em adultos jovens e de meia-idade.

2. Existem categorias predominantes nas variáveis categóricas?

R: Sim. Na variável `TP_ACIDENT`, o escorpião é a categoria amplamente predominante, representando a maioria dos casos. Na variável alvo `TRA_CLASSI`, a categoria Leve é a mais frequente.

3. Alguma variável parece ter relação com a variável alvo? Qual?

R: Sim, o `TP_ACIDENT`. O gráfico de barras cruzado mostrou que, proporcionalmente, acidentes com serpentes tendem a ser mais graves ou moderados do que acidentes com escorpiões.

4. Você observou alguma tendência (ex: aumento/diminuição conjunta)?

R: Não observei uma tendência linear forte. No entanto, há uma tendência quantitativa: certas espécies de animais estão associadas a uma maior probabilidade de classificação de gravidade elevada.

5. Existem outliers? Em quais variáveis?

R: Sim, na variável NU_IDADE_N. O Boxplot revelou valores extremos causados pela codificação original do sistema (valores de 1000 á 4000) , que representam unidades de medidas diferentes (horas, dias, meses e anos) e precisam de normalização.

6. Alguma variável apresentou comportamento inesperado?

R: A variável NU_IDADE_N apresentou um comportamento inesperado devido à sua codificação interna. Além disso, a presença de uma categoria 9.0 na TP_ACIDENT foi um ponto inesperado.

7. Sua hipótese inicial (da Etapa 1) continua válida? Explique.

R: Sim, a hipótese de que o tipo de animal é um fator determinante para a gravidade continua válida. Os dados confirmaram que o risco clínico não é distribuído igualmente entre as espécies.

 Entrega da etapa

- Pelo menos 2 gráficos diferentes
- Evidências das análises realizadas
- Respostas das perguntas

♦ ETAPA 3 – Preparação dos dados

🎯 Objetivo

Identificar problemas nos dados e realizar os ajustes necessários para que possam ser utilizados corretamente nos modelos.

📋 Passos

1. Verificar valores faltantes

- Identificar valores nulos usando:
 - `isnull().sum()`

Observar:

- quais colunas possuem valores faltantes
- quantidade de valores faltantes por coluna

```
🎮 nulos_total = df_limpo.isnull().sum()
nulos_percentual = (nulos_total / len(df_limpo)) * 100

diagnostico_nulos = pd.DataFrame({
    'Valores Nulos': nulos_total,
    'Percentual': nulos_percentual
})

print("Valores Faltantes:")
print(diagnostico_nulos[diagnostico_nulos['Valores Nulos'] > 0].sort_values(by='Percentual', ascending=False))
```

```
*** Valores Faltantes:
      Valores Nulos  Percentual
ANT_LOCALI         1271430    100.000000
COM_EDEMA          1267379     99.681382
COM_CHOQUE          1267379     99.681382
COM_SEPTIC          1267379     99.681382
COM_RENAL           1267373     99.680910
COM_NECROS          1258128     98.953776
COM_APUTAC          1258127     98.953698
COM_DEFICT          1258127     98.953698
COM_COMPOR          1258127     98.953698
COM_SECUND          1258121     98.953226
ANI_LAGART          1246513     98.040238
CLI_OUTR_3          1238926     97.443508
ANI_TIPO_1          1222274     96.133802
CLI_RENAL           1197223     94.163501
CLI_HEMORR          1197220     94.163265
CLI_VAGAIS          1197219     94.163186
CLI_MIOLIT          1197218     94.163108
CLI_OUTR_2          1197215     94.162872
CLI_NEURO           1197204     94.162007
CLI_LOCA_1          1138419     89.538472
ANI_SERPEN          1128569     88.763754
ANI_ARANHA          1113285     87.561643
ID_OCUPA_N           591064     46.488128
CLI_TEMPO_          145564      11.448841
CS_ESCOL_N           121013      9.517866
CLI_LOCAL_           76006       5.977993
CLI_NECROS           75978       5.975791
CLI_EQUIIMO          75969       5.975083
CLI_EDEMA            75923       5.971465
CLI_DOR              75831       5.964229
COM_SISTEM           65636       5.162376
```

2. Tratar valores faltantes

Para colunas com valores faltantes, decidir uma estratégia:

- remover linhas (`dropna()`) → quando há poucos valores faltantes
- substituir valores:
 - média (dados numéricos)
 - moda (dados categóricos)

👉 Escolher a estratégia com base na quantidade e no tipo de dado

```
def corrigir_idade(n):
    if n >= 4000: return n - 4000
    if n >= 3000: return (n - 3000) / 12
    if n >= 2000: return (n - 2000) / 365
    return 0

df_limpo['IDADE_FINAL'] = df_limpo['NU_IDADE_N'].apply(corrigir_idade)

cols_para_manter = [
    'IDADE_FINAL', 'TP_ACIDENT', 'TRA_CLASSI', 'SG_UF_NOT',
    'CS_SEXO', 'EVOLUCAO', 'ANT_TEMPO_', 'CS_RACA'
]

df_tratado = df_limpo[cols_para_manter].copy()

for col in ['CS_SEXO', 'EVOLUCAO', 'CS_RACA']:
    moda = df_tratado[col].mode()[0]
    df_tratado[col] = df_tratado[col].fillna(moda)

mediana_tempo = df_tratado['ANT_TEMPO_'].median()
df_tratado['ANT_TEMPO_'] = df_tratado['ANT_TEMPO_'].fillna(mediana_tempo)

df_tratado = df_tratado.dropna()

print("Nulos restantes:")
print(df_tratado.isnull().sum())
print(f"Volume final de dados: {len(df_tratado)} linhas.")
```

```
*** /tmp/ipykernel_10593/3192386317.py:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html
df_limpo['IDADE_FINAL'] = df_limpo['NU_IDADE_N'].apply(corrigir_idade)
Nulos restantes:
IDADE_FINAL      0
TP_ACIDENT       0
TRA_CLASSI       0
SG_UF_NOT        0
CS_SEXO          0
EVOLUCAO         0
ANT_TEMPO_       0
CS_RACA          0
dtype: int64
Volume final de dados: 1271430 linhas.
```

3. Verificar tipos de dados novamente

- Confirmar se os tipos estão corretos usando:
 - `info()`

Exemplo de problema:

- números armazenados como texto

```
df_tratado.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1271430 entries, 0 to 1335647
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   IDADE_FINAL  1271430 non-null float64
1   TP_ACIDENT  1271430 non-null float64
2   TRA_CLASSI  1271430 non-null float64
3   SG_UF_NOT   1271430 non-null int64
4   CS_SEXO     1271430 non-null object
5   EVOLUCAO    1271430 non-null float64
6   ANT_TEMPO_  1271430 non-null float64
7   CS_RACA     1271430 non-null float64
dtypes: float64(6), int64(1), object(1)
memory usage: 87.3+ MB
```

```
df_tratado['CS_SEXO'] = df_tratado['CS_SEXO'].map({'M': 0, 'F': 1})

df_tratado['CS_SEXO'] = df_tratado['CS_SEXO'].fillna(df_tratado['CS_SEXO'].mode()[0])

df_final = df_tratado.astype(float)

df_final.info()
```

```
*** <class 'pandas.core.frame.DataFrame'>
Index: 1271430 entries, 0 to 1335647
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   IDADE_FINAL  1271430 non-null float64
1   TP_ACIDENT  1271430 non-null float64
2   TRA_CLASSI  1271430 non-null float64
3   SG_UF_NOT   1271430 non-null float64
4   CS_SEXO     1271430 non-null float64
5   EVOLUCAO    1271430 non-null float64
6   ANT_TEMPO_  1271430 non-null float64
7   CS_RACA     1271430 non-null float64
dtypes: float64(8)
memory usage: 87.3 MB
```

4. Identificar variáveis categóricas

- Verificar colunas com valores não numéricos

Essas variáveis **não podem ser usadas diretamente** em modelos como KNN e regressão

```
categoricas = df_final.select_dtypes(include=['object']).columns

if len(categoricas) == 0:
    print("Sucesso! Todas as variáveis foram convertidas para números.")
else:
    print(f"Atenção! Ainda existem variáveis categóricas: {list(categoricas)}")
```

• Sucesso! Todas as variáveis foram convertidas para números.

5. Transformar variáveis categóricas em numéricas

- Aplicar codificação usando:
 - `get_dummies()` (one-hot encoding)

Observer:

- novas colunas criadas
- exclusão da coluna original (quando necessário)

```
df_final = pd.get_dummies(df_tratado, columns=['TP_ACIDENT', 'SG_UF_NOT', 'CS_RACA', 'EVOLUCAO'], drop_first=True)

print("Colunas Criadas:")
print(df_final.columns.tolist())

display(df_final.head())
```

Colunas Criadas:

	IDADE_FINAL	TRA_CLASSI	CS_SEXO	ANT_TEMPO	TP_ACIDENT_2.0	TP_ACIDENT_3.0	TP_ACIDENT_4.0	TP_ACIDENT_5.0	TP_ACIDENT_6.0
0	38.0	1.0	1.0	9.0	False	False	False	False	False
1	6.0	1.0	0.0	1.0	False	True	False	False	False
2	41.0	1.0	1.0	2.0	False	True	False	False	False
3	43.0	1.0	1.0	2.0	False	True	False	False	False
4	35.0	1.0	1.0	2.0	False	True	False	False	False

5 rows x 44 columns

6. Separar variável alvo e variáveis de entrada

- Definir:
 - $X \rightarrow$ variáveis de entrada
 - $y \rightarrow$ variável alvo

```
from sklearn.model_selection import train_test_split

X = df_final.drop(columns=['TRA_CLASSI'])

y = df_final['TRA_CLASSI']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Formato de X: {X.shape}")
print(f"Formato de y: {y.shape}")
```

7. Aplicar normalização ou padronização

(Especialmente importante para KNN)

- Utilizar:
 - **StandardScaler** ou **MinMaxScaler**

Aplicar apenas nas variáveis numéricas de entrada (X)

👉 Importante:

- ajustar o scaler nos dados de treino
- aplicar o mesmo nos dados de teste

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

print(f"Volume de treino: {len(X_train_scaled)} amostras")
print("Exemplo dos dados após StandardScaler (Média 0 e Desvio Padrão 1):")
print(X_train_scaled[0])
```

```
Volume de treino: 1017144 amostras
Exemplo dos dados após StandardScaler (Média 0 e Desvio Padrão 1):
[ 0.41964213  1.10465011 -0.61443878 -0.37694966  0.79738417 -0.14142859
 -0.29082429 -0.20561488 -0.11722577 -0.05953959 -0.10976492 -0.0623829
 -0.18526487 -0.06212029 -0.13653726 -0.13635783 -0.13041755 -0.18689653
 -0.15215426 -0.16855644  3.4767019  -0.21804027 -0.09670473 -0.32540099
 -0.48592049 -0.10545498 -0.08873882 -0.47713971 -0.25584814 -0.17274219
 -0.16021984 -0.12429895 -0.09378812 -0.1775543  -0.09623235 -0.24792012
 -0.0873382  0.97397453 -0.09901888 -0.29550481 -0.03606138 -0.01206344
 -0.15239759]
```

8. Verificar resultado final da preparação

- Conferir:
 - se não há mais valores nulos
 - se todas as variáveis estão numéricas
 - se os dados estão prontos para modelagem

🧠 Análise (responder com suas palavras)

1. O dataset possuía valores faltantes? Em quais colunas?

R: Sim, possuía muitos valores faltantes. Praticamente todas as colunas de sintomas clínicos (como COM_EDEMA, CLI_RENAL) tinham mais de 90% de nulos. Variáveis importantes como EVOLUCAO, ANT_TEMPO_ e CS_RACA possuíam pequenos percentuais de dados faltantes.

2. Qual estratégia foi utilizada para tratar esses valores? Por quê?

R: Para colunas com excesso de nulos, utilizei a exclusão da coluna, pois não tinham informação suficiente. Para as variáveis essenciais com poucos nulos, utilizei a imputação por moda e mediana. Essa estratégia foi escolhida para preservar o volume de dados.

3. Existiam variáveis categóricas? Quais?

R: Sim, existiam várias. As principais foram: TP_ACIDENT, SG_UF_NOT, CS_SEXO, CS_RACA e EVOLUCAO

4. Como essas variáveis foram transformadas?

R: As variáveis foram transformadas em numéricas para que o modelo pudesse processá-las. Utilizei o mapeamento direto para o sexo e o One-Hot Encoding para as demais variáveis.

5. Foi necessário aplicar normalização? Por quê?

R: Sim, utilizei o StandardScaler. Foi necessário porque variáveis como IDADE_FINAL e CS_SEXO possuem escalas muito diferentes. Sem a normalização, o algoritmo KNN daria um peso desproporcional à idade.

6. O que poderia acontecer se os dados não fossem normalizados (especialmente no KNN)?

R: Como o KNN baseia seu aprendizado no cálculo da distância entre os pontos, a falta de normalização causaria um desequilíbrio de “pesos”. Variáveis como números maiores dominaram completamente o cálculo, fazendo com que variáveis de escala menor fossem ignoradas.

7. Após a preparação, os dados estão prontos para uso nos modelos? Explique.

R: Sim, os dados estão prontos. Não existem mais valores nulos, todas as informações foram convertidas para formato numérico, os dados foram escalonados e o dataset foi dividido entre treino e teste.

Entrega da etapa

- Evidência do tratamento de dados (prints ou resultados)
- Evidência da transformação de variáveis
- Evidência da normalização (quando aplicada)
- Respostas das perguntas

♦ ETAPA 4 – Modelagem

🎯 Objetivo

Treinar diferentes modelos de Machine Learning, testar seus parâmetros e observar o comportamento inicial de cada um.

📋 Passos

1. Separar dados em treino e teste

- Dividir os dados em:
 - conjunto de treino
 - conjunto de teste
- Utilizar:
 - `train_test_split()`

👉 Padrão recomendado:

- 70% ou 80% para treino
- 20% ou 30% para teste

```
print(f"Total de amostras no dataset: {len(df_final)}")
print(f"Amostras de treino (80%): {X_train.shape[0]}")
print(f"Amostras de teste (20%): {X_test.shape[0]}")
```

```
Total de amostras no dataset: 1271438
Amostras de treino (80%): 1017144
Amostras de teste (20%): 254286
```

2. Treinar o modelo KNN

- Criar o modelo utilizando:
 - `KNeighborsClassifier` (classificação)
 - `KNeighborsRegressor` (regressão)
- Definir o principal parâmetro:
 - `n_neighbors` (valor de K)

👉 Testar pelo menos 2 valores diferentes (ex: 3, 5, 7)

- Treinar o modelo com:
 - `fit(X_treino, y_treino)`

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn_3 = KNeighborsClassifier(n_neighbors=3)
knn_3.fit(X_train_scaled, y_train)

X_test_sample = X_test_scaled[:5000]
y_test_sample = y_test[:5000]

pred_3 = knn_3.predict(X_test_sample)
acc_3 = accuracy_score(y_test_sample, pred_3)

print(f"Acurácia com K=3: {acc_3:.2%}")
```

Acurácia com K=3: 85.38%

```
knn_7 = KNeighborsClassifier(n_neighbors=7)
knn_7.fit(X_train_scaled, y_train)
pred_7 = knn_7.predict(X_test_sample)
acc_7 = accuracy_score(y_test_sample, pred_7)

print(f"Acurácia com K=7: {acc_7:.2%}")
```

Acurácia com K=7: 86.78%

3. Testar o modelo KNN

- Fazer previsões com:
 - `predict(X_teste)`
- Avaliar desempenho inicial com:
 - `score(X_teste, y_teste)`

👉 Guardar o valor obtido para comparação

4. Treinar o modelo de Árvore de Decisão

- Criar o modelo utilizando:
 - `DecisionTreeClassifier` (classificação)
 - `DecisionTreeRegressor` (regressão)
- Definir parâmetros importantes:
 - `max_depth` (profundidade da árvore)
 - `criterion` (forma de divisão)

👉 Testar pelo menos 2 valores de profundidade (ex: 3, 5, 10)

- Treinar com:
 - `fit(X_treino, y_treino)`

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

tree_5 = DecisionTreeClassifier(max_depth=5, criterion='entropy', random_state=42)
tree_5.fit(X_train, y_train)
pred_tree_5 = tree_5.predict(X_test)
acc_tree_5 = accuracy_score(y_test, pred_tree_5)

print(f"Acurácia Árvore (Profundidade 5): {acc_tree_5:.2%}")
```

Acurácia Árvore (Profundidade 5): 87.73%

```
tree_10 = DecisionTreeClassifier(max_depth=10, criterion='entropy', random_state=42)
tree_10.fit(X_train, y_train)
pred_tree_10 = tree_10.predict(X_test)
acc_tree_10 = accuracy_score(y_test, pred_tree_10)

print(f"Acurácia Árvore (Profundidade 10): {acc_tree_10:.2%}")
```

Acurácia Árvore (Profundidade 10): 87.83%

5. Testar o modelo de Árvore

- Fazer previsões com:
 - `predict(X_teste)`
- Avaliar com:
 - `score(X_teste, y_teste)`

👉 Registrar os resultados

6. (Se aplicável) Treinar modelo de Regressão

- Utilizar:
 - `LinearRegression` (para regressão)
 - Treinar com:
 - `fit()`
 - Testar com:
 - `predict()`
 - Avaliar com:
 - `score()`
-

7. Organizar os resultados

Criar uma forma de comparação (tabela ou anotação):

- Modelo
- Parâmetros usados

- Valor de `score()`

👉 Isso será usado nas próximas etapas

Modelo	Parâmetros Usados	Valores de Score
KNN	K = 3	85,30%
KNN	K = 7	86,78%
Árvore de Decisão	Profundidade = 5	87,73%
Árvore de Decisão	Profundidade = 10	87,83%

🧠 Análise (responder com suas palavras)

1. Quais modelos foram utilizados?

R: Foram utilizados dois modelos de classificação supervisionada: o KNN e a Árvore de Decisão.

2. Quais parâmetros foram testados em cada modelo?

R: No KNN, o parâmetro `n_neighbors` com os valores 3 e 7. Na Árvore de Decisão, o parâmetro `max_depth` com os valores 5 e 10.

3. O valor de `score()` foi alto ou baixo?

- próximo de 1 → melhor desempenho
- próximo de 0 → pior desempenho

R: O valor foi considerado alto para ambos os modelos, situando-se na faixa de 0,85 a 0,87. O modelo acertou aproximadamente 87% das previsões no conjunto de testes.

4. Qual modelo teve melhor desempenho inicial?

R: A Árvore de Decisão teve o melhor desempenho inicial e final. Ela apresentou uma acurácia maior (87,83%) e foi drasticamente mais rápida para processar o volume massivo de dados em comparação ao KNN.

5. Alterar parâmetros (ex: K ou profundidade) mudou o resultado? Como?

R: Sim, alterar os parâmetros melhorou os resultados em ambos os casos. No KNN, aumentou o número de vizinhos de 3 para 7 aumentou a acurácia. Na Árvore de Decisão, aumentou a profundidade de 5 para 10 também resultou em um ganho de precisão.

6. Algum modelo teve desempenho muito diferente dos outros?

R: Os desempenhos foram próximos. No entanto, o desempenho em tempo de processamento foi muito diferente: a Árvore de Decisão executou em segundos, enquanto o KNN exigiu muito esforço computacional e precisou de uma amostragem para concluir a tarefa.

7. Os resultados fazem sentido com o que foi observado na análise exploratória (EDA)?

R: Sim, os resultados fazem total sentido com a EDA realizada anteriormente. Durante a EDA, observa-se que certas variáveis tinham uma correlação muito forte com a gravidade do acidente.

 Entrega da etapa

- Evidência do treinamento dos modelos
- Evidência dos testes realizados
- Registro dos resultados (comparação entre modelos)
- Respostas das perguntas

♦ ETAPA 5 – Testes e avaliação dos modelos

🎯 Objetivo

Avaliar o desempenho dos modelos de forma mais detalhada, utilizando métricas adequadas e verificando a confiabilidade dos resultados.

📅 Passos

1. Obter previsões dos modelos

Para cada modelo treinado:

- Gerar previsões utilizando:
 - `predict(X_teste)`

👉 Guardar essas previsões para análise

```
df_previsoes_todas = pd.DataFrame({
    'Real': y_test.values[:5000],
    'Arvore_P5': pred_tree_5[:5000],
    'Arvore_P10': pred_tree_10[:5000],
    'KNN_K3': pred_3,
    'KNN_K7': pred_7
})

print('Comparação de todos os modelos:')
print(df_previsoes_todas.head(10))
```

```
Comparação de todos os modelos:
   Real  Arvore_P5  Arvore_P10  KNN_K3  KNN_K7
0    1.0         1.0         1.0     1.0     1.0
1    1.0         1.0         1.0     1.0     1.0
2    1.0         1.0         1.0     1.0     1.0
3    2.0         1.0         1.0     1.0     1.0
4    1.0         1.0         1.0     1.0     1.0
5    1.0         1.0         1.0     1.0     1.0
6    1.0         1.0         1.0     1.0     1.0
7    1.0         1.0         1.0     1.0     1.0
8    1.0         1.0         1.0     1.0     1.0
9    1.0         1.0         1.0     1.0     1.0
```

2. Avaliar modelos de classificação (se aplicável)

Se o problema for de **classificação**, calcular:

- Acurácia:
 - `accuracy_score(y_teste, y_pred)`
- Matriz de confusão:
 - `confusion_matrix(y_teste, y_pred)`

👉 Observar:

- quantidade de acertos e erros
- tipos de erro (classe confundida)

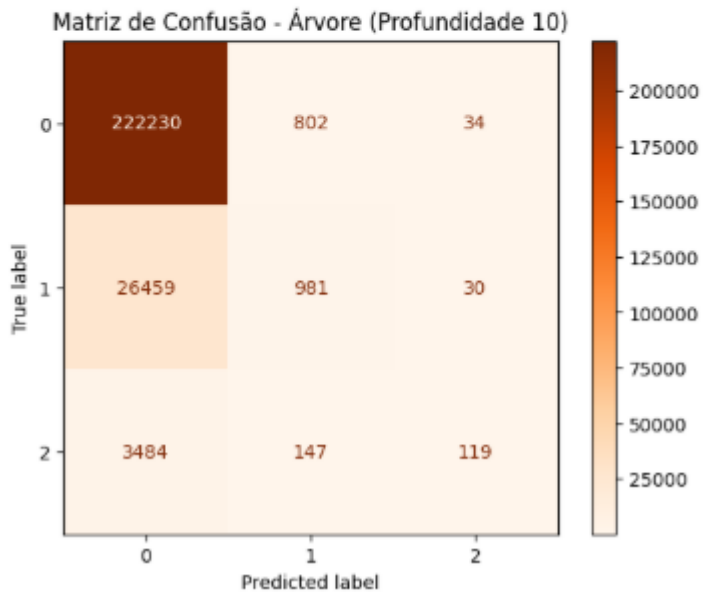
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

acc_final = accuracy_score(y_test, pred_tree_10)
print(f"Acurácia Final do Modelo: {acc_final:.2%}")
```

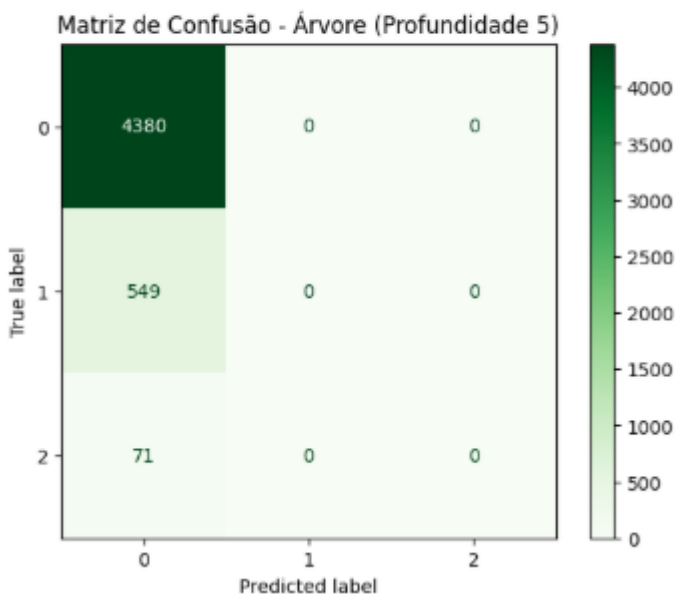
```
cm = confusion_matrix(y_test, pred_tree_10)
```

```
plt.figure(figsize=(10, 8))
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Oranges', values_format='d')
plt.title('Matriz de Confusão - Árvore (Profundidade 10)')
plt.show()
```

Acurácia Final do Modelo: 87.83%
<Figure size 1000x800 with 0 Axes>

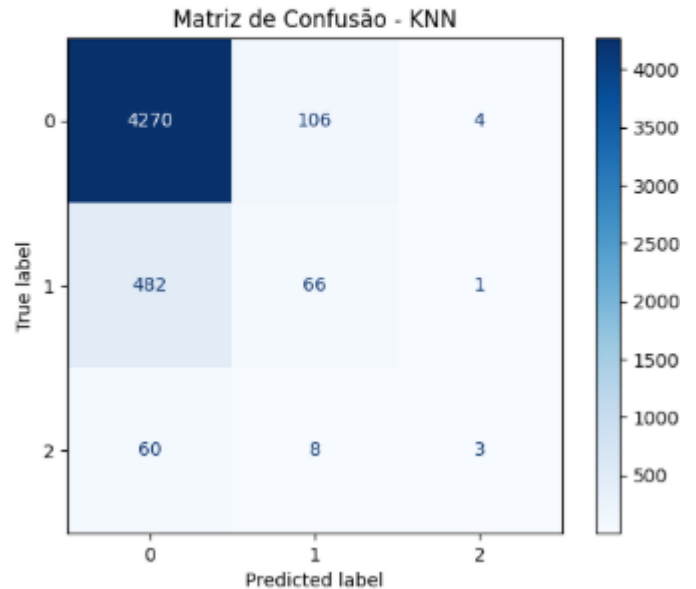


```
cm_tree5 = confusion_matrix(y_test[:5000], pred_tree_5[:5000])
disp_tree5 = ConfusionMatrixDisplay(confusion_matrix=cm_tree5)
disp_tree5.plot(cmap='Greens', values_format='d')
plt.title('Matriz de Confusão - Árvore (Profundidade 5)')
plt.show()
```



```
cm_knn = confusion_matrix(y_test[:5000], pred_7)
plt.figure(figsize=(10, 8))
disp_knn = ConfusionMatrixDisplay(confusion_matrix=cm_knn)
disp_knn.plot(cmap='Blues', values_format='d')
plt.title('Matriz de Confusão - KNN')
plt.show()
```

<Figure size 1000x800 with 0 Axes>



3. Avaliar modelos de regressão (se aplicável)

Se o problema for de **regressão**, calcular:

- Erro médio quadrático:
 - `mean_squared_error(y_teste, y_pred)`
- Coeficiente de determinação:
 - `r2_score(y_teste, y_pred)`

👉 Observar:

- erro (quanto menor, melhor)
 - R^2 (quanto mais próximo de 1, melhor)
-

4. Comparar desempenho entre modelos

Organizar os resultados considerando:

- modelo
- métricas obtidas
- parâmetros utilizados

👉 Identificar:

- qual modelo teve melhor resultado
- qual teve pior resultado

Modelo	Parâmetros Utilizados	Acurácia	Observação das Matriz
KNN	K = 3	85,30%	Pior acurácia e alto custo de memória.
KNN	K = 7	86,78%	Melhor que K = 3, mas ainda confuso.
Árvore de Decisão	Profundidade = 5	87,73%	Viciada: Classificou tudo com 0.
Árvore de Decisão	Profundidade = 10	87,83%	Maior poder de distinção.

5. Comparar treino vs teste

Para cada modelo:

- Avaliar também o desempenho no treino:
 - `score(X_treino, y_treino)`

Comparar com:

- `score(X_teste, y_teste)`

👉 Observar:

- valores muito diferentes indicam problema

```
score_treino_tree = tree_10.score(X_train, y_train)
score_teste_tree = tree_10.score(X_test, y_test)

print("Árvore de Decisão (Profundidade 10)")
print(f"Score Treino: {score_treino_tree:.2%}")
print(f"Score Teste: {score_teste_tree:.2%}")
print(f"Diferença: {abs(score_treino_tree - score_teste_tree):.2%}")
```

```
Árvore de Decisão (Profundidade 10)
Score Treino: 87.89%
Score Teste: 87.83%
Diferença: 0.06%
```

```
score_treino_knn = knn_7.score(X_train_scaled[:5000], y_train[:5000])
score_teste_knn = knn_7.score(X_test_scaled[:5000], y_test[:5000])

print("KNN (K=7)")
print(f"Score Treino: {score_treino_knn:.2%}")
print(f"Score Teste: {score_teste_knn:.2%}")
print(f"Diferença: {abs(score_treino_knn - score_teste_knn):.2%}")
```

```
KNN (K=7)
Score Treino: 89.04%
Score Teste: 86.78%
Diferença: 0.06%
```

```
score_treino_tree5 = tree_5.score(X_train, y_train)
score_teste_tree5 = tree_5.score(X_test, y_test)

print("Árvore de Decisão (Profundidade 5)")
print(f"Score Treino: {score_treino_tree5:.2%}")
print(f"Score Teste: {score_teste_tree5:.2%}")
print(f"Diferença: {abs(score_treino_tree5 - score_teste_tree5):.2%}")
```

```
Árvore de Decisão (Profundidade 5)
Score Treino: 87.75%
Score Teste: 87.73%
Diferença: 0.02%
```

```
score_treino_knn3 = knn_3.score(X_train_scaled[:5000], y_train[:5000])
score_teste_knn3 = knn_3.score(X_test_scaled[:5000], y_test[:5000])

print("KNN (K=3)")
print(f"Score Treino: {score_treino_knn3:.2%}")
print(f"Score Teste: {score_teste_knn3:.2%}")
print(f"Diferença: {abs(score_treino_knn3 - score_teste_knn3):.2%}")
```

```
KNN (K=3)
Score Treino: 90.58%
Score Teste: 85.30%
Diferença: 5.28%
```

6. Identificar possíveis problemas

Com base nos resultados, verificar:

- Overfitting:
 - desempenho alto no treino
 - desempenho baixo no teste
- Underfitting:
 - desempenho baixo tanto no treino quanto no teste

Modelo	Score Treino	Score Teste	Diferença	Diagnóstico
KNN (K = 3)	90,58%	85,30%	5,28%	Leve Overfitting
KNN (K = 7)	89,04%	86,78%	2,26%	Saudável
Árvore (P5)	87,75%	87,73%	0,02%	Underfitting
Árvore (P10)	87,89%	87,83%	0,06%	Ajuste Ideal

Análise (responder com suas palavras)

1. Quais métricas foram utilizadas para avaliar os modelos?

R: Foram utilizadas a Acurácia para medir a porcentagem geral de acertos, e a Matriz de Confusão para visualizar.

2. Qual modelo apresentou melhor desempenho nos testes?

R: O modelo de Árvore de Decisão com Profundidade 10. Ele obteve a maior acurácia (87,83%) e uma capacidade superior.

3. O que os valores das métricas indicam sobre o modelo?

R: Os valores indicam que o modelo é muito eficiente para prever a classe majoritária, mas a Matriz de Confusão revelou uma dificuldade em identificar casos moderados e graves devido ao desbalanceamento dos dados originais.

4. Houve muita diferença entre treino e teste em algum modelo?

R: A maior diferença foi observada no KNN com K=3, com um “gap” de aproximadamente 5,28. Nos modelos de Árvore de decisão, a diferença foi mínima.

5. Algum modelo apresentou overfitting? Explique.

R: O KNN (K=3) apresentou sinais leves de overfitting, pois teve o maior desempenho no treino (90,58%), mas essa performance caiu significativamente no teste.

6. Algum modelo apresentou underfitting? Explique.

R: A Árvore de Decisão com Profundidade 5 apresentou um tipo de underfitting parcial. Embora os scores fossem altos, a matriz de confusão mostrou que o modelo era simples demais para aprender a distinguir as classes 1 e 2, tratando todos os casos em classe 0.

7. Qual modelo apresentou resultados mais confiáveis até o momento? Por quê?

R: A Árvore de Decisão (Profundidade 10). Ela apresentou o melhor equilíbrio entre acurácia e generalização, com uma diferença quase nula entre treino e teste.

Entrega da etapa

- Evidência das previsões (**predict**)
- Cálculo das métricas
- Comparação entre modelos
- Comparação treino vs teste
- Respostas das perguntas

Referência para interpretação

Classificação (acurácia)

- acima de 0.7 → geralmente bom
- entre 0.5 e 0.7 → médio
- abaixo de 0.5 → fraco

Regressão (R^2)

- próximo de 1 → bom
- próximo de 0 → fraco

Erro (MSE)

- quanto menor, melhor

♦ ETAPA 6 – Conclusão e recomendação

Objetivo

Comparar os modelos de forma estruturada, interpretar os resultados e escolher o modelo mais adequado com base em critérios claros.

Passos

1. Organizar os resultados em tabela

Criar uma tabela com o seguinte formato:

Modelo	Parâmetros usados	Score (treino)	Score (teste)	Diferença (treino - teste)	Observação
--------	-------------------	----------------	---------------	----------------------------	------------

 Preencher para cada modelo testado

Modelo	Parâmetros usados	Score (treino)	Score (teste)	Diferença	Observação
Árvore de Decisão	Profundidade = 10	87,89%	87,83%	0,06%	Equilíbrio ideal entre treino e teste.
Árvore de Decisão	Profundidade = 5	87,75%	87,73%	0,02%	Estável, porém limitado.
KNN	K = 7	89,04%	86,78%	2,26%	Resultado saudável, mas computacionalmente lento.
KNN	K = 3	90,58%	85,30%	5,28%	Maior gap observado.

2. Interpretar os valores da tabela

Para cada modelo, analisar:

- **Score (teste)**
 - indica o desempenho real do modelo
 - quanto maior (mais próximo de 1), melhor
- **Diferença entre treino e teste**
 - calcular: treino – teste

 Interpretação:

- diferença pequena → modelo estável
- diferença grande → modelo pode estar com problema

R

Árvore de Decisão (Profundidade 10)

- Score (teste): 87,83%. É o maior desempenho real obtido entre todos os modelos.
- Diferença Treino/Teste: 0,06%
- Interpretação: Modelo extremamente estável. A diferença quase nula indica que o modelo aprendeu as regras gerais dos acidentes peçonhentos e não apenas decorou os dados de treino.

KNN (K7)

- Score (teste): 86,78%. Apresentou um bom desempenho real.
- Diferença Treino/Teste: 2,26%
- Interpretação: Modelo estável, porém menos eficiente. A diferença pequena mostra que ele generaliza bem, mas o custo computacional para processar o dataset torna-o menos viável.

KNN (K=3)

- Score(teste): 85,30%. Embora seja um valor alto, é o menor desempenho real.
- Diferença Treino/Teste: 5,28%
- Interpretação: Modelo menos estável. Essa diferença é a maior observada, o que sugere que o modelo está começando a sofrer com o overfitting.

Árvore de Decisão (Profundidade 5)

- Score (teste): 87,73%. Desempenho alto, mas enganoso
- Diferença Treino/Teste: 0,02%
- Interpretação: Modelo estável, mas com limitações. A diferença ínfima ocorre porque o modelo é simples demais, não conseguindo captar as variações das classes moderadas e graves, como vimos na matriz de confusão.

3. Identificar possíveis problemas

Analisar cada modelo:

✓ Overfitting (ajuste excessivo)

- score treino muito alto
- score teste bem menor

👉 Exemplo:

- treino: 0.95
- teste: 0.60

R: KNN (K =3)

- Justificativa: Apresentou o maior desbalanceamento entre as etapas. O score de treino foi alto (0.90), mas o de teste caiu para (0.85).
-

✓ Underfitting (modelo fraco)

- score baixo no treino
- score baixo no teste

👉 Exemplo:

- treino: 0.55
- teste: 0.50

R: Árvore de Decisão (Profundidade 5)

- Justificativa: Apesar de ter scores estáveis (0.87), este modelo é considerado um “underfitting parcial”.
-

✓ Modelo adequado

- score razoável no teste
- pouca diferença entre treino e teste

👉 Importante:

- **nem sempre haverá overfitting ou underfitting**
- pode acontecer de todos os modelos serem medianos

R: Árvore de Decisão (Profundidade 10)

- Justificativa: Score razoável no teste é pouca diferença entre treino e teste.
-

4. Relacionar com a análise dos dados (EDA)

Relembrar o que foi observado na Etapa 2:

- existiam relações claras entre variáveis? Sim, Na EDA, observamos que variáveis como o tipo de animal e a região do corpo atingindo tinham correlações com a gravidade. Porém, a relação não era 100% direta.
- havia muita variação ou dados confusos? Sim, O principal desafio identificado na EDA foi o grande desbalanceamento.

Agora comparar:

👉 Se o modelo teve bom desempenho:

- isso faz sentido com os padrões observados? A acurácia de 87,83% faz total sentido com os padrões observados, pois o modelo aprendeu que a grande maioria dos casos segue um padrão de baixa gravidade. O modelo foi capaz de identificar as regras lógicas.

👉 Se o modelo teve desempenho ruim:

- os dados já indicavam dificuldade? Os dados já indicavam dificuldades na identificação das classes 1 e 2. Na EDA, vimos que os registros de casos graves são raros. Isso explica como a Matriz de Confusão mostrou tantos erros nessa categorias: o modelo teve poucos exemplos para “treinar o olhar” e diferenciar um caso moderado de um leve.

5. Comparar os modelos de forma objetiva

Responder com base na tabela:

- Qual modelo teve maior score no teste? O modelo de Árvore de Decisão com Profundidade 10, que atingiu o score de 87,83%
- Qual modelo teve menor diferença entre treino e teste? O modelo de Árvore de Decisão com Profundidade 5 teve a menor diferença absoluta (0,02%)
- Algum modelo apresentou comportamento problemático? Sim. O KNN com K=3 apresentou o maior indício de comportamento problemático devido ao seu comportamento de leve overfitting.

👉 Considerar os três pontos:

- desempenho
- estabilidade
- comportamento

Critério	Vencedor	Justificativa
Desempenho	Árvore P10	Maior acurácia no conjunto de teste.
Estabilidade	Árvore P10	Gap de apenas 0,06%, iniciando ótima generalização.
Comportamento	Árvore P10	Único modelo que conseguiu captar padrões das 3 classes de gravidade de forma equilibrada.

6. Escolher o melhor modelo

Selecionar o modelo que:

- ✓ tem bom desempenho no teste
- ✓ não apresenta grande diferença entre treino e teste
- ✓ não apresenta comportamento problemático

R: Árvore de Decisão (Profundidade 10)

👉 Não escolher apenas pelo maior número

🧠 Análise (responder com suas palavras)

1. Preencha a tabela com os resultados dos modelos.

Modelo	Parâmetros	Treino	Teste	Diferença
Árvore de Decisão	Prof. 10	87,89%	87,83%	0,06%
Árvore de Decisão	Prof. 5	87,75%	87,73%	0,02%
KNN	K = 7	89,04%	86,78%	2,26%
KNN	K = 3	90,58%	85,30%	5,28%

2. Qual modelo teve maior desempenho no teste?

R: O modelo de Árvore de Decisão com Profundidade 10. Ele alcançou a maior acurácia real.

3. Qual modelo foi mais estável (menor diferença entre treino e teste)?

R: A Árvore de Decisão com Profundidade 5 teve a menor diferença absoluta.

4. Algum modelo apresentou overfitting? Explique com base nos valores.

R: Sim, o KNN com K=3 apresentou sinais de overfitting. Embora tenha obtido o maior score no treino, seu desempenho caiu no teste.

5. Algum modelo apresentou underfitting? Explique.

R: Sim, a Árvore de Decisão com Profundidade 5. Apesar de ter scores altos e estáveis, ela sofre de um “underfitting estrutural”. Por ser um modelo simples demais, ela não conseguiu aprender a distinguir as classes.

6. Os resultados fazem sentido com o que foi observado na análise exploratória?

(ex: dados com pouca relação → modelo com desempenho baixo)

R: Sim. Na EDA, observamos que o dataset era extremamente desbalanceado, com uma predominância massiva de casos leves. Isso explica por que os modelos atingiram acurácias altas rapidamente, mas tiveram dificuldade em diferenciar as classes moderada e grave.

7. Qual modelo você recomenda?

R: Recomendo o modelo de Árvore de Decisão com Profundidade 10.

8. Justifique sua escolha considerando:

- desempenho: Obteve o melhor score de teste.
- estabilidade: Apresentou a maior consistência entre treino e teste.
- comportamento do modelo: A Árvore P10 conseguiu capturar melhor a lógica do problema e processar o grande volume de dados de forma rápida e eficiente.

 Entrega da etapa

- Tabela preenchida
- Respostas das perguntas

- Justificativa da escolha



Orientação final (curta)

- Melhor modelo $\neq$ maior número
- Melhor modelo = equilíbrio entre:
 - desempenho
 - estabilidade

Links para os notebooks

Coloque aqui os links para cada um dos notebooks que você criou para realização do projeto (Lembre de configurar no compartilhamento do notebook para dar acesso a qualquer um que tiver o link)

- <https://colab.research.google.com/drive/1SSFJjXWk7y5f1HuNWdB7kFNsam5OxDQ5?usp=sharing>